HASHING
PROBLEM 6

QUESTION:

DS	**Session**	Hashing	**Question Information**	Level 1 Challenge 96

Problem Description

Jenish and Neha are excellent friends. After performing several queries, Neha challenges Jenish to determine the highest possible Rating of the provided array A. According to Neha, the highest occurrence of an element in an array is its rating.

Jenish is given M and Q, the Magical Numbers. Jenish may perform Addition or Subtraction with M at most Q times for each element in the supplied Array A.

Because Jenish is stumped and unable to discover a solution, assist him in determining the highest possible ratings for the given array after applying queries to each element.

Constraints :

- $1 \leq N \leq 1000000$
- $1 \leq M \leq 100$
- $1 \leq Q \leq 10$
- $1 \leq A_i \leq 1000000$

Input :

- First line of Input contains integer M
- Second line contains integer Q
- Third line contains integer N
- Fourth line contains N elements representing elements of array A

Output :

- Output the highest possible rating of array A after applying Queries.

Explanation for test case 1

Jenish can add 1 in 1st element and subtract 1 from 3rd element to get highest frequency of 2 , i.e. 3

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
1
1
4
1 2 3 4
```

EXPECTED OUTPUT

```
3
```

Test Case 2

INPUT (STDIN)

```
1
1
5
11 12 31 41 51
```

EXPECTED OUTPUT

```
2
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=0;i<N;i++)
```

Test Case 2

KEYWORD

```
hash[A[i]]++;
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
9
```

Test Case 2

TOKEN COUNT

```
300
```

Test Case 3

NLOC

```
48
```

Code Editor c

```c
#include <stdio.h>
int main()
```

Custom Input (stdin) T1 T2

SOURCE CODE:

```c
#include <stdio.h>
#include <stdlib.h>

int int_asc_compare(const void* a, const void* b) {
    return (*(const int*)a - *(const int*)b);
}

int main() {
    int step_val, max_steps, count;
    if (scanf("%d %d %d", &step_val, &max_steps, &count) != 3) return 0;

    int elements[count];
    for (int i = 0; i < count; i++) {
        scanf("%d", &elements[i]);
    }

    qsort(elements, count, sizeof(int), int_asc_compare);

    int max_rating_score = 1;
    long long reach_window = 2LL * step_val * max_steps;
    int left_ptr = 0;

    for (int right_ptr = 0; right_ptr < count; right_ptr++) {
        while ((long long)elements[right_ptr] - elements[left_ptr] > reach_window) {
            left_ptr++;
        }
        int current_group = right_ptr - left_ptr + 1;
        if (current_group > max_rating_score) {
            max_rating_score = current_group;
        }
    }

    printf("%d\n", max_rating_score);
    return 0;
}
```

OUTPUT:

```
1
2
4
1 2 3 4
4
```

```
1
1
5
11 12 31 41 51
2
3
```